

Electronics & Computer Science

COMP 1324 Algorithmics

Tutorial 8

Note: Question 7 is somewhat challenging and most probably will not be covered in the tutorial. A solution will be provided after the tutorial.

1. A delivery company has n trucks and a **single** team to load the trucks. Let f_i be the time needed by the team to load truck i and t_i be the time needed by truck i to deliver its content to the required destination *after* it is loaded. In other words, the trucks are loaded **sequentially** according to some schedule but once loaded a truck starts delivery independently of the others. Let T be the time when all the trucks have delivered their contents. Devise a greedy schedule for loading the trucks to minimize T . (Note that the team loads the trucks **one at a time**)
2. You are given the value of a certain stock at regular intervals in an array S . $S[0]$ is the initial value of the stock, $S[i]$ is the value at time i . At time i you are allowed to purchase maximum i shares of the stock for a total cost of $i \cdot S[i]$. Given an initial sum K , devise a greedy algorithm to purchase the largest number of stocks.
3. A certain monetary system has bills of values: 1,5,10,20. Given an amount x you need to find the smallest number of bills (repetition is allowed) that add up to the requested amount x .
 - (a) Give a greedy algorithm that finds the smallest number of bills that add up to x .
 - (b) Show that, by providing a counter example (specific value of x), the greedy approach no longer finds the optimal solution if an additional bill of value 25 is added to the system.
 - (c) Devise a dynamic programming strategy to find the optimal solution by writing down the recursive formula in terms of subproblems.
4. You need to climb a staircase with n steps. Each time you can move 1 or 2 steps. Let $c[n]$ be the number of ways you can climb the stairs. Obtain a recursive formula for $c[n]$. For example, if there are 3 steps you can climb the stairs in three different ways by taking: three 1-steps, one 1-step followed by one 2-steps, one 2-steps followed by one 1-step.
5. Consider a grid of dimension $n \times n$. A robot needs to go from cell $(0, 0)$ to $(n - 1, n - 1)$. At each step the robot can move right or down (not left or up). Write a function that returns the total number of paths that the robot can take.
6. You are supposed to manage the construction of billboards on a highway. The allowable positions, from some arbitrary starting point, are set by the government as $\{x_1, x_2, \dots, x_n\}$ with the restriction that billboards have to be at least 5 km apart. The profit from each billboard is given by the numbers $\{v_1, v_2, \dots, v_n\}$. For example, $X = \{8, 10, 12, 15, 16, 17, 20\}$ and $V = \{2, 7, 14, 9, 3, 13, 8\}$ then the optimal solution is to construct two billboard at positions 12 and 17 for a total profit of 27.
 - (a) Give a dynamic programming algorithm to find the optimized profit.
 - (b) Give an algorithm that prints the set of positions corresponding to the solution.
7. Consider a **variation** of the knapsack problem (with replacement) where an item can be added to the knapsack **more than once**.

- (a) Given n items with values v_i and weights w_i , $i = 1 \dots n$ and a knapsack with capacity of C , obtain a recursive formula between the optimal solution and its subproblems.
- (b) Give the solution table for the instance $v = [2, 3, 4, 6]$, $w = [2, 3, 4, 5]$ with $C = 13$. Compare the solution with the 0-1 knapsack problem (with no replacement).
8. A single processor needs to perform n tasks. Each task i has a duration t_i , a deadline d_i and only if the task is completed by the deadline it earns a profit p_i , otherwise the profit is zero. As an example, consider the following:

Name	Duration	Deadline	Profit
A	1	2	100
B	1	1	19
C	1	2	27
D	1	1	25
E	1	3	15

Clearly in the above example, to earn the maximum profit you would schedule task A from 0 to 1 then task C from 1 to 2 and finally task E from 2 to 3 for a total earning of 142. The above case is simple, in that all durations are 1, and a greedy strategy where the highest profit is selected first works. When the durations are different than 1, the greedy approach doesn't always work. For example, consider the following:

Name	Duration	Deadline	Profit
A	3	8	100
B	2	6	59
C	1	9	40
D	2	5	70
E	4	10	101
F	5	10	140

In the above example, a greedy strategy would give $\{F, E\}$ as solution with a profit of 241. The optimal solution, obtained using dynamic programming, is $\{C, D, A, E\}$ for a profit of 311.

Using dynamic programming to solve the general problem. (Hint: start by sorting the tasks by deadline, in ascending order)